

PoRE Lab6 实验报告

姓名：徐锦云

学号：23307130447

耗时：10小时

Task 1：混淆技术实践（40%）

1. 使用 EncodeLiterals 对 main 函数进行混淆（5%）

混淆后的运行与验证

在实验目录执行：

```
gcc -o run_enc_lit obf_encode_literals.c -w
printf 'quicksort\n3\n3 1 2\n' | ./run_enc_lit
```

```
# pore @ pore in ~/pore26/pore_23307130447/Lab6 on git:master x [12:34:33]
$ gcc -o run_enc_lit obf_encode_literals.c -w

# pore @ pore in ~/pore26/pore_23307130447/Lab6 on git:master x [12:35:32]
$ printf 'quicksort\n3\n3 1 2\n' | ./run_enc_lit
Please enter the command ('quicksort' to sort an array): Enter the number of elements: Enter 3 integers:
Original array:
3 1 2
Sorted array:
1 2 3
```

这说明混淆之后照样能正确执行，混淆正确。

a) 使用的 tigress 命令

```
tigress --Seed=47 --Environment=x86_64:Linux:Gcc:13.3 \
--Transform=InitEntropy --Functions=main --InitEntropyKinds=vars \
--Transform=InitOpaque --Functions=main --InitOpaqueStructs=env \
--Transform=EncodeLiterals --Functions=main \
Lab6_Task_1_23307130447.c --out=obf_encode_literals.c
```

(InitEntropy / InitOpaque 为 EncodeLiterals 官方示例中的常见前置步骤，用于准备熵与不透明环境。)

b) 混淆后的 main 函数

字符串常量被编码，运行时通过 `_TIG_EL....stringEncoder` 还原；比较与返回值等处可配合熵变量做等价变换：

```
/* BEGIN FUNCTION-DEF main LOC=UNKNOWN */
int main(int _TIG_IZ_LzNX_formal_argc , char **_TIG_IZ_LzNX_formal_argv , char
**_TIG_IZ_LzNX_formal_envp )
{
    char command[20] ;
    int arr[100] ;
```

```

int n ;
char *realcommand ;
int tmp ;

{
{
{
{
{
_TIG_Iz_LzNX_envp = (char **)0;
goto _TIG_Iz_LzNX_envp__INITINLINE__TIG_Iz_LzNX_envp__INIT;
}
_TIG_Iz_LzNX_envp__INITINLINE__TIG_Iz_LzNX_envp__INIT: /* CIL Label */ ;
}
{
{
_TIG_Iz_LzNX_argv = (char **)0;
goto _TIG_Iz_LzNX_argv__INITINLINE__TIG_Iz_LzNX_argv__INIT;
}
_TIG_Iz_LzNX_argv__INITINLINE__TIG_Iz_LzNX_argv__INIT: /* CIL Label */ ;
}
{
{
_TIG_Iz_LzNX_argc = 0;
goto _TIG_Iz_LzNX_argc__INITINLINE__TIG_Iz_LzNX_argc__INIT;
}
_TIG_Iz_LzNX_argc__INITINLINE__TIG_Iz_LzNX_argc__INIT: /* CIL Label */ ;
}
goto megaInitINLINE_megaInit;
}
megaInitINLINE_megaInit: /* CIL Label */ ;
}
while (1) {
    __asm__ volatile ("##_ANNOTATION_INITIALREGION_-TIG-Iz-LzNX--0");
    _TIG_Iz_LzNX_argc = _TIG_Iz_LzNX_formal_argc;
    _TIG_Iz_LzNX_argv = _TIG_Iz_LzNX_formal_argv;
    _TIG_Iz_LzNX_envp = _TIG_Iz_LzNX_formal_envp;
    break;
}
_TIG_EL_LzNX_3_stringEncoder(0, _TIG_EL_LzNX_3_encodeStrings_litStr0);
#line 62 "Lab6_Task_1_23307130447.c"
printf((char const /* __restrict */)(_TIG_EL_LzNX_3_encodeStrings_litStr0));
_TIG_EL_LzNX_3_stringEncoder(1, _TIG_EL_LzNX_3_encodeStrings_litStr1);
#line 63
scanf((char const /* __restrict */)(_TIG_EL_LzNX_3_encodeStrings_litStr1), command);
_TIG_EL_LzNX_3_stringEncoder(2, _TIG_EL_LzNX_3_encodeStrings_litStr2);
#line 64
realcommand = _TIG_EL_LzNX_3_encodeStrings_litStr2;
#line 65
tmp = strcmp((char const *)(command), (char const *)realcommand);
#line 65
if (tmp == 0L + (int)(_TIG_iE_LzNX_1_alwaysZero * (_TIG_iE_LzNX_1_entropy ^ 9) + 0)) {
#line 66

```

```

        getUserInput(arr, & n);
        _TIG_EL_LzNX_3_stringEncoder(3, _TIG_EL_LzNX_3_encodeStrings_litStr3);
#line 68
        printf((char const /* __restrict */)(_TIG_EL_LzNX_3_encodeStrings_litStr3));
#line 69
        printArray(arr, n);
#line 71
        quickSort(arr, 0L + (int )(_TIG_iE_LzNX_1_alwaysZero * (_TIG_iE_LzNX_1_entropy + 2) +
0),
                n - (1L + (int )(_TIG_iE_LzNX_1_alwaysZero * (_TIG_iE_LzNX_1_entropy + 2) +
0)));
        _TIG_EL_LzNX_3_stringEncoder(4, _TIG_EL_LzNX_3_encodeStrings_litStr4);
#line 73
        printf((char const /* __restrict */)(_TIG_EL_LzNX_3_encodeStrings_litStr4));
#line 74
        printArray(arr, n);
    } else {
        _TIG_EL_LzNX_3_stringEncoder(5, _TIG_EL_LzNX_3_encodeStrings_litStr5);
#line 76
        printf((char const /* __restrict */)(_TIG_EL_LzNX_3_encodeStrings_litStr5));
    }
#line 79
    return (0L + (int )(_TIG_iE_LzNX_1_alwaysZero * (_TIG_iE_LzNX_1_entropy - 7) + 0));
}
}

/* END FUNCTION-DEF main LOC=UNKNOWN */

/* BEGIN FUNCTION-DEF partition LOC=UNKNOWN */
int partition(int *arr , int low , int high )
{
    int pivot ;
    int i ;
    int j ;

    {
#line 14
        pivot = *(arr + high);
#line 15
        i = low - 1;
#line 17
        j = low;
#line 17
        while (j <= high - 1) {
#line 18
            if (*(arr + j) < pivot) {
#line 19
                i ++;
#line 20
                swap(arr + i, arr + j);
            }
#line 17
            j ++;
        }

```

```

#line 23
    swap(arr + (i + 1), arr + high);
#line 24
    return (i + 1);
}
}
/* END FUNCTION-DEF partition LOC=UNKNOWN */

/* BEGIN FUNCTION-DEF swap LOC=UNKNOWN */
void swap(int *a , int *b )
{
    int temp ;

    {
#line 7
        temp = *a;
#line 8
        *a = *b;
#line 9
        *b = temp;
#line 10
        return;
    }
}
/* END FUNCTION-DEF swap LOC=UNKNOWN */

```

2. 使用 Flatten 对 partition 函数进行混淆 (5%)

混淆后的运行与验证

```

gcc -o run_flat obf_flatten_partition.c -w
printf 'quicksort\n3\n3 1 2\n' | ./run_flat

```

```

# pore @ pore in ~/pore26/pore_23307130447/Lab6 on git:master x [12:35:42]
$ gcc -o run_flat obf_flatten_partition.c -w

# pore @ pore in ~/pore26/pore_23307130447/Lab6 on git:master x [12:38:13]
$ printf 'quicksort\n3\n3 1 2\n' | ./run_flat
Please enter the command ('quicksort' to sort an array): Enter the number of elements: Enter 3 integers:
Original array:
3 1 2
Sorted array:
1 2 3

```

这说明混淆之后照样能正确执行，混淆正确。

a) 使用的 tigress 命令

```

tigress --Seed=47 --Environment=x86_64:Linux:Gcc:13.3 \
  --Transform=Flatten --Functions=partition \
  Lab6_Task_1_23307130447.c --out=obf_flatten_partition.c

```

b) 混淆后的 partition 函数

控制流被展开为 `while(1){ switch(_TIG_FN_..._next) { ... } }` 形式:

```
/* BEGIN FUNCTION-DEF partition LOC=UNKNOWN */
int partition(int *arr , int low , int high )
{
    int pivot ;
    int i ;
    int j ;
    unsigned long _TIG_FN_fBpX_1_partition_next ;

    {
    {
        _TIG_FN_fBpX_1_partition_next = 4UL;
    }
    while (1) {
        switch (_TIG_FN_fBpX_1_partition_next) {
            case 4UL: ;
            {
                _TIG_FN_fBpX_1_partition_next = 0UL;
            }
            break;
            case 8UL: ;
            return (i + 1);
            break;
            case 1UL: ;
            if (*(arr + j) < pivot) {
                {
                    _TIG_FN_fBpX_1_partition_next = 9UL;
                }
            } else {
                {
                    _TIG_FN_fBpX_1_partition_next = 7UL;
                }
            }
            break;
            case 9UL:
#line 19
                i ++;
#line 20
                swap(arr + i, arr + j);
                {
                    _TIG_FN_fBpX_1_partition_next = 7UL;
                }
                break;
            case 6UL:
#line 23
                swap(arr + (i + 1), arr + high);
                {
                    _TIG_FN_fBpX_1_partition_next = 8UL;
                }
                break;
        }
    }
}
```

```

        case 0UL:
#line 14
        pivot = *(arr + high);
#line 15
        i = low - 1;
#line 17
        j = low;
        {
            _TIG_FN_fBpX_1_partition_next = 2UL;
        }
        break;
        case 7UL:
#line 17
        j ++;
        {
            _TIG_FN_fBpX_1_partition_next = 2UL;
        }
        break;
        case 2UL: ;
        if (j <= high - 1) {
            {
                _TIG_FN_fBpX_1_partition_next = 1UL;
            }
        } else {
            {
                _TIG_FN_fBpX_1_partition_next = 6UL;
            }
        }
        break;
        default:
        break;
    }
}
}

/* END FUNCTION-DEF partition LOC=UNKNOWN */

```

3. 使用 EncodeArithmetic 对 partition 函数进行混淆 (5%)

混淆后的运行与验证

```

gcc -o run_ea obf_encode_arith_partition.c -w
printf 'quicksort\n3\n3 1 2\n' | ./run_ea

```

```
# pore @ pore in ~/pore26/pore_23307130447/Lab6 on git:master x [12:43:00]
$ gcc -o run_ea obf_encode_arith_partition.c -w

# pore @ pore in ~/pore26/pore_23307130447/Lab6 on git:master x [12:47:15]
$ printf 'quicksort\n3\n3 1 2\n' | ./run_ea
Please enter the command ('quicksort' to sort an array): Enter the number of elements: Enter 3 integers:
Original array:
3 1 2
Sorted array:
1 2 3
```

这说明混淆之后照样能正确执行，混淆正确。

a) 使用的 tigress 命令

```
tigress --Seed=47 --Environment=x86_64:Linux:Gcc:13.3 \
--Transform=EncodeArithmetic --Functions=partition \
Lab6_Task_1_23307130447.c --out=obf_encode_arith_partition.c
```

b) 混淆后的 partition 函数

算术运算被改写为按位组合形式，语义与原始 `+`、`-`、`比较`、`自增` 等价：

```
/* BEGIN FUNCTION-DEF partition LOC=UNKNOWN */
int partition(int *arr , int low , int high )
{
    int pivot ;
    int i ;
    int j ;

    {
    {
#line 14 "Lab6_Task_1_23307130447.c"
        pivot = *(arr + high);
#line 15
        i = (low & ~ 1) - (~ low & 1);
#line 17
        j = low;
    }
#line 17
        while (((unsigned int)((~ (j ^ (high - 1)) >> 1) + (j & ~ (high - 1))) >> 31U) & 1) {
#line 18
            if (((unsigned int)((*(arr + j) & ~ pivot) | (~ (*(arr + j) ^ pivot) & (*(arr + j) -
pivot)))) >> 31U) & 1) {
                {
#line 19
                    i = ((i ^ ~ 1) + ((i | 1) << 1)) + 1;
#line 20
                    swap(arr + i, arr + j);
                }
            }
#line 17
            j = ((j ^ ~ 1) + ((j | 1) << 1)) + 1;
        }
    }
}
```

```
#line 23
    swap(arr + (i + 1), arr + high);
#line 24
    return ((i | 1) + (i & 1));
}
}
/* END FUNCTION-DEF partition LOC=UNKNOWN */
```

4. 使用 AddOpaque 对 partition 函数进行混淆 (5%)

混淆后的运行与验证

```
gcc -o run_ao obf_addopaque_partition.c -w
printf 'quicksort\n3\n3 1 2\n' | ./run_ao
```

```
# pore @ pore in ~/pore26/pore_23307130447/Lab6 on git:master x [12:48:56]
$ gcc -o run_ao obf_addopaque_partition.c -w

# pore @ pore in ~/pore26/pore_23307130447/Lab6 on git:master x [12:50:03]
$ printf 'quicksort\n3\n3 1 2\n' | ./run_ao
Please enter the command ('quicksort' to sort an array): Enter the number of elements: Enter 3 integers:
Original array:
3 1 2
Sorted array:
1 2 3
Focus folder in explorer (ctrl + click)
```

这说明混淆之后照样能正确执行，混淆正确。

a) 使用的 tigress 命令

```
tigress --Seed=47 --Environment=x86_64:Linux:Gcc:13.3 \
  --Transform=InitEntropy --Functions=main --InitEntropyKinds=vars \
  --Transform=InitOpaque --Functions=partition --InitOpaqueStructs=env \
  --Transform=AddOpaque --Functions=partition \
  --AddOpaqueCount=50 \
  --AddOpaqueSplitBasicBlocks=true \
  --AddOpaqueKinds=call,bug,true,junk,question \
  Lab6_Task_1_23307130447.c --out=obf_addopaque_partition.c
```

b) 混淆后的 partition 函数

`partition` 主体被插入大量不透明谓词与辅助副本函数调用，体积显著增大；以下为入口附近示意：

```
/* BEGIN FUNCTION-DEF partition LOC=UNKNOWN */
int partition(int *arr , int low , int high )
{
    int pivot ;
    int i ;
    int j ;

    {
    {
```



```

{
    if ((unsigned int )(_TIG_iE_XfZk_1_alwaysZero * (_TIG_iE_XfZk_1_entropy | 9) + 1)) {
        {
            if ((unsigned int )(_TIG_iE_XfZk_1_alwaysZero * (_TIG_iE_XfZk_1_entropy * 1) + 0)) {
                {
                    while (1) {
                        if ((unsigned int )(_TIG_iE_XfZk_1_alwaysZero * (_TIG_iE_XfZk_1_entropy | 10) + 0) >
8U) {
                            {
                                {
                                    {
                                        {
                                            {
                                                {

                                }
                                }
                                }
                                }
                                }
                                }
                            }
                        } else {
                            break;
                        }
                    }
                    {
                        {
                            while (1) {
                                if ((unsigned int )(_TIG_iE_XfZk_1_alwaysZero * (_TIG_iE_XfZk_1_entropy * 3) + 1)
== 9U) {
                                    {
                                        {
                                            {
                                                {
                                                    {
                                                        {

                                }
                                }
                                }
                                }
                                }
                                }
                            }
                        } else {
                            break;
                        }
                    }
                    while (1) {
                        {

                            if (low >= _TIG_iE_XfZk_1_entropy - 8) {
                                partition_QUESTION_9_16(& low, & high, & i, & j);
                            } else {
                                partition_QUESTION_9_16_COPY(& high, & i, & low, & j, 1L);
                            }
                        }
                    }
                }
            }
        }
    }
}

```

[illegible]

```

{
{
{
{
partition_QUESTION_3_4_COPY(low);
}
}
}
}
}
}
}
}
{
{
{
{
if ((unsigned int )(_TIG_iE_Xfzk_1_alwaysZero * (_TIG_iE_Xfzk_1_entropy | 1) + 1)) {
{
{

}
}
}
} else {

}
}
}
}
}
}
{
{
{
{
{
{
{
{
{
{
{
}
}
}
}
}
}
}
}
}
}
if (low <= (_TIG_iE_Xfzk_1_entropy ^ 3)) {
{
{
{
{
if ((unsigned int )(_TIG_iE_Xfzk_1_alwaysZero * (_TIG_iE_Xfzk_1_entropy | 2) + 0))

```

```

{

```

```

{
    while (1) {
        while (1) {
            partition_QUESTION_3_5(& arr + 1, & low + 1, & high - 1, & pivot - -1,
                                   & i + -1, & j - -1);

        }
    }
}
} else {
    {
    {
    {
    partition_QUESTION_3_5(& arr, & low, & high, & pivot, & i, & j);
    }
    }
    }
}
} else {
    {
    if ((unsigned int )(_TIG_iE_xfZk_1_alwaysZero * (_TIG_iE_xfZk_1_entropy + 5) + 1))
{
        {
        {
        {
        {
        {
        partition_QUESTION_3_5_COPY(& low, & arr, & high, 0, & pivot, & i, & j);
        }
        }
        }
        }
        }
        }
        } else {
            {
            {
            {
            {
            {
            __asm__ volatile (".byte 0x97,0x1");
            }
            }
            }
            }
            }
        }
    }
}
}

```

```

{
{
{
{
{
{

}
}
}
}
}
}
}
}
}
}
} else {
    while (1) {
        {
            if ((unsigned int )(_TIG_iE_Xfzk_1_alwaysZero * (_TIG_iE_Xfzk_1_entropy | 7) + 0))
{
            while (1) {
                {
                    {
                    {
                    {
                    {
                    {

                }
                }
                }
                }
                }
                }
            }
        } else {
            {
            {
            {
            {
            {
            {
            {

            }
            }
            }
            }
            }
            }
        }
    }
}
}

```

```

{
{
while (1) {
    if (((low >= (_TIG_iE_XfZk_1_entropy | 4)) == low) > low) {
        {
        {
        {
        {
        {
        {
        {

        }
        }
        }
        }
        }
        }
        }
    } else {
        break;
    }
    {
    {
    {
    {
    {
    {
    {
    {
    partition_QUESTION_2_2_COPY((& j + -1) + -1, (& i - -1) + -1, (& low + -1) - 1,
                                (double)4.43308543176f, (& high + -1) + -1,
                                (& arr - 1) - -1, (& pivot - 1) + 1);

    }
    }
    }
    }
    }
    }
    }
}
}
}
{
{
{
{
{
{
{
{
{
}
}
}
}
}
}
}

```

```

    }
    }
    }
    }
}
}
}
} else {
{
{
{
{
{
{
{
{

}
}
}
}
}
}
}
}
if (low >= (_TIG_iE_Xfzk_1_entropy | 4)) {
{
{
    if ((unsigned int )(_TIG_iE_Xfzk_1_alwaysZero * (_TIG_iE_Xfzk_1_entropy | 2) + 0)) {
        {
            {
                {
                    {
                        {
                            {
                                getUserInput(arr, arr);
                            }
                        }
                    }
                }
            }
        }
    } else {
        {
            {
                {
                    {
                        {
                            {
                                }
                            }
                        }
                    }
                }
            }
        }
    }
}

```



```

    }
    }
    }
    }
    }
    }
    }
    } else {
        while (1) {
            {
                if ((unsigned int )(_TIG_iE_Xfzk_1_alwaysZero * (_TIG_iE_Xfzk_1_entropy ^ 6) +
0)) {

                    } else {
                        {
                            while (1) {
                                if ((unsigned int )(_TIG_iE_Xfzk_1_alwaysZero * (_TIG_iE_Xfzk_1_entropy +
10) + 0) == 0U) {
                                    {
                                        {
                                            {
                                                {
                                                    {
                                                        {

                                                    }
                                                    }
                                                    }
                                                    }
                                                    }
                                                    }
                                                    }
                                                } else {
                                                    break;
                                                }
                                            {
                                                {
                                                    {
                                                        {
                                                            {
                                                                {
                                                                    {
                                                                        {
                                                                            partition_QUESTION_2_2((& arr - 1) + 1, (& low + 1) - -1, (& high - 1) -
-1,
                                                                    (& pivot + -1) + 1, (& i + -1) + -1, (& j - -1) +
-1);
                                                                }
                                                            }
                                                        }
                                                    }
                                                }
                                            }
                                        }
                                    }
                                }
                            }
                        }
                    }
                }
            }
        }
    }
}

```



```

        if ((unsigned int )(_TIG_iE_Xfzk_1_alwaysZero * (_TIG_iE_Xfzk_1_entropy + 10) +
0) > 30) {
            {
                {
                    {
                        {
                            {
                                {
                                    }
                                }
                            }
                        }
                    }
                }
            }
        } else {
            {
                if ((unsigned int )(_TIG_iE_Xfzk_1_alwaysZero * (_TIG_iE_Xfzk_1_entropy | 4) +
1)) {
                    {
                        {
                            {
                                {
                                    {
                                        {
                                            }
                                        }
                                    }
                                }
                            }
                        }
                    }
                } else {
                    {
                        {
                            {
                                {
                                    {
                                        {
                                            {
                                                printArray(arr, i);
                                            }
                                        }
                                    }
                                }
                            }
                        }
                    }
                }
            }
        }
    }
}

```



```

    }
    }
    }
    }
    }
    }
    {
if ((unsigned int )(_TIG_iE_XfZk_1_alwaysZero * (_TIG_iE_XfZk_1_entropy ^ 10) +
0)) {
    {
    {
    {
    {
    {
    {
    high = main(i, 0, 0);
    }
    }
    }
    }
    }
    }
    } else {
    {
    {
    {
    {
    {
    {

    }
    }
    }
    }
    }
    }

    }
    {
while (1) {
    while (1) {
        {
            if (((low >= (_TIG_iE_XfZk_1_entropy | 4)) == low) > low) >= high) {
                return (low);
            } else {
                {
                {
                {
                {
                {
                {

```



```

{
{
{
{
{

}
}
}
}
}
}
}
}
while (1) {
{
while (1) {
{
if ((unsigned int )(_TIG_iE_Xfzk_1_alwaysZero * (_TIG_iE_Xfzk_1_entropy * 8)
+ 0)) {

{
{
{
{
{
{
{
__asm__ volatile (".byte 0xba,0x8e,0x16,0xd,0xe");
}
}
}
}
}
}
}
}
} else
if (((low >= (_TIG_iE_Xfzk_1_entropy | 4)) == low) < low) {
{
{
{
{
{
{
{

}
}
}
}
}
}
}
} else {
break;

```



```

{
}
}
}
}
}
}
}
}
}
{
{
if ((unsigned int )(_TIG_iE_XfZk_1_alwaysZero * (_TIG_iE_XfZk_1_entropy - 5) + 0)) {
{

}
} else {
{
{
{
{
if ((unsigned int )(_TIG_iE_XfZk_1_alwaysZero * (_TIG_iE_XfZk_1_entropy + 6) + 0)) {
{
{
{
{
__asm__ volatile (".byte 0x48,0x55,0xb5,0xe8,0x8,0xd9,0x40");
}
}
}
}
}
}
} else {
{
{
{
{
{
{
}
}
}
}
}
}
}
}
}
if (low >= _TIG_iE_XfZk_1_entropy * 2) {
{
if ((unsigned int )(_TIG_iE_XfZk_1_alwaysZero * (_TIG_iE_XfZk_1_entropy * 10) + 1)) {
{
{

```

```

{
{
{
{
{
partition_QUESTION_2_3(& arr, & high, & pivot, & i, & j);
}
}
}
}
}
}
}
}
} else {
{
{
{
{
{
{
{
}
}
}
}
}
}
}
}
} else {
{
{
if ((unsigned int )(_TIG_iE_XfZk_1_alwaysZero * (_TIG_iE_XfZk_1_entropy ^ 9) + 1)) {
    if ((unsigned int )(_TIG_iE_XfZk_1_alwaysZero * (_TIG_iE_XfZk_1_entropy + 5) + 0)) {
        {
            if ((unsigned int )(_TIG_iE_XfZk_1_alwaysZero * (_TIG_iE_XfZk_1_entropy + 2) + 1))
{
                {
                {
                {
                if ((unsigned int )(_TIG_iE_XfZk_1_alwaysZero * (_TIG_iE_XfZk_1_entropy | 4) +
0)) {
                    {
                    {
                    {
                    {

                }
                }
                }
                }
            }

```

```

    } else {
        {
        {
        {
        {

        }
        }
        }
        }
        }
    }
    }
    }
    }
} else {
    {
    {
    {
    {
    {
    {
    {
    swap(arr, arr);
    }
    }
    }
    }
    }
    }
    }
}
{
{
{
{
{
{
{
{
__asm__ volatile (".byte 0xc5");
}
}
}
}
}
}
}
}
} else {
    {
    {

```

```

    if ((unsigned int )(_TIG_iE_Xfzk_1_alwaysZero * (_TIG_iE_Xfzk_1_entropy * 3) + 0))
    {
        {
            {
                {
                    {
                        {
                            getUserInput(arr, arr);
                        }
                    }
                }
            }
        }
    } else {
        {
            {
                {
                    {
                        {
                            }
                        }
                    }
                }
            }
        }
    }
    {
        {
            if ((unsigned int )(_TIG_iE_Xfzk_1_alwaysZero * (_TIG_iE_Xfzk_1_entropy ^ 3) + 1))
            {
                {
                    {
                        {
                            }
                        }
                    }
                }
            } else {
                {
                    {
                        {
                            high = partition(arr, pivot, pivot);
                        }
                    }
                }
            }
        }
    }
}

```



```
} else {
    {
    {
    {
    {
    {
    {
    {
    }
    }
    }
    }
    }
    }
} else {
{
{
{
{
{
{
__asm__ volatile (".byte 0x5,0x4b,0x3f,0x0,0x0,0x1,0x0,0x0");
}
}
}
}
}
}
}
if ((unsigned int )(_TIG_iE_xfZk_1_alwaysZero * (_TIG_iE_xfZk_1_entropy | 4) + 1)) {
{
if ((unsigned int )(_TIG_iE_xfZk_1_alwaysZero * (_TIG_iE_xfZk_1_entropy + 6) + 0)) {
{
{
{
{
pivot = main(j, 0, 0);
}
}
}
}
}
```

```

    } else {
        {
        {
        {
        {

        }
        }
        }
        }
        }
    }
    {
    if ((unsigned int )(_TIG_iE_XfZk_1_alwaysZero * (_TIG_iE_XfZk_1_entropy - 6) + 1)) {
        if ((unsigned int )(_TIG_iE_XfZk_1_alwaysZero * (_TIG_iE_XfZk_1_entropy + 4) + 1)) {
#line 24
            return (i + 1);
        } else {
#line 24
            return (1);
        }
    } else {
        {
        {
        __asm__ volatile (".byte 0x96,0x7c,0x35,0x1d:");
        }
        }
    }
    }
    }
} else {
    {
    if ((unsigned int )(_TIG_iE_XfZk_1_alwaysZero * (_TIG_iE_XfZk_1_entropy | 3) + 1)) {
        {
        {
        {
        {
        {
        {
        {
        __asm__ volatile (".byte 0x0,0xe8,0x7c,0x30,0x0:");
        }
        }
        }
        }
        }
        }
        }
    } else {
        {
        {
        {

```


b) 混淆后的代码（全部代码）

- 完整可编译源码文件： `obf_merge.c`。
- 合并后的统一函数名（由 Tigress 自动生成）：
`_TIG_MG_Mh2G_1_getUserInput_partition_printArray_quickSort_swap`，通过额外参数 `whichBlock__7` 区分原各函数逻辑。

核心结构节选（`whichBlock__7` 分派）：

```
void _TIG_MG_Mh2G_1_getUserInput_partition_printArray_quickSort_swap(void *tigressRetVal ,
                                                                    int *arr__0 ,
                                                                    int *size__1 ,
                                                                    int low__3 ,
                                                                    int high__4 ,
                                                                    int whichBlock__7 )
{
    // ...
    if (whichBlock__7 == 0) {
        /* getUserInput */
    } else
    if (whichBlock__7 == 1) {
        /* partition */
    } else
    if (whichBlock__7 == 2) {
        /* printArray */
    } else
    if (whichBlock__7 == 3) {
        /* quickSort */
    } else
    if (whichBlock__7 == 4) {
        /* swap */
    } else {

    }
}
```

6. 对源码一次性使用全部上述 5 种混淆技术（10%）

将第 1 ~ 5 步的变换按单次 `tigress` 调用串联（顺序：

`EncodeLiterals`→`Flatten`→`EncodeArithmetic`→`AddOpaque`→`Merge`；`Merge` 仍仅包含五个业务函数）。

混淆后的运行与验证

```
gcc -o run_all5 obf_all5.c -w
printf 'quicksort\n3\n3 1 2\n' | ./run_all5
```

```
# pore @ pore in ~/pore26/pore_23307130447/Lab6 on git:master x [12:55:39]
$ gcc -o run_all5
Focus folder in explorer (ctrl + click)
# pore @ pore in ~/pore26/pore_23307130447/Lab6 on git:master x [12:57:37]
$ printf 'quicksort\n3\n3 1 2\n' | ./run_all5
Please enter the command ('quicksort' to sort an array): Enter the number of elements: Enter 3 integers:
Original array:
3 1 2
Sorted array:
1 2 3
```

这说明混淆之后照样能正确执行，混淆正确。

a) 使用的 tigress 命令

```
tigress --Seed=47 --Environment=x86_64:Linux:Gcc:13.3 \
--Transform=InitEntropy --Functions=main --InitEntropykinds=vars \
--Transform=InitOpaque --Functions=main,partition --InitOpaqueStructs=env \
--Transform=EncodeLiterals --Functions=main \
--Transform=Flatten --Functions=partition \
--Transform=EncodeArithmetic --Functions=partition \
--Transform=AddOpaque --Functions=partition \
--AddOpaqueCount=50 \
--AddOpaqueSplitBasicBlocks=true \
--AddOpaquekinds=call,bug,true,junk,question \
--Transform=Merge --Functions=partition,quickSort,swap,printArray,getUserInput \
Lab6_Task_1_23307130447.c --out=obf_all5.c
```

b) 混淆后的代码（全部代码）

完整可编译源码文件： obf_all5.c。

代码太多无法复制，和上面那个一起作为附件提交。下面随便展示一些代码：

```
/* BEGIN FUNCTION-DEF main LOC=UNKNOWN */
int main(int _TIG_IZ_V2zB_formal_argc , char **_TIG_IZ_V2zB_formal_argv , char
**_TIG_IZ_V2zB_formal_envp )
{
    char command[20] ;
    int arr[100] ;
    int n ;
    char *realcommand ;
    int tmp ;

    {
    {
    {
    {
    {
    _TIG_IZ_V2zB_envp = (char **)0;
    goto _TIG_IZ_V2zB_envp__INITINLINE__TIG_IZ_V2zB_envp__INIT;
    }
    _TIG_IZ_V2zB_envp__INITINLINE__TIG_IZ_V2zB_envp__INIT: /* CIL Label */ ;
    }
}
```

```

{
{
_TIG_Iz_V2zB_argv = (char **)0;
goto _TIG_Iz_V2zB_argv__INITINLINE__TIG_Iz_V2zB_argv__INIT;
}
_TIG_Iz_V2zB_argv__INITINLINE__TIG_Iz_V2zB_argv__INIT: /* CIL Label */ ;
}
{
{
_TIG_Iz_V2zB_argc = 0;
goto _TIG_Iz_V2zB_argc__INITINLINE__TIG_Iz_V2zB_argc__INIT;
}
_TIG_Iz_V2zB_argc__INITINLINE__TIG_Iz_V2zB_argc__INIT: /* CIL Label */ ;
}
goto megaInitINLINE_megaInit;
}
megaInitINLINE_megaInit: /* CIL Label */ ;
}
while (1) {
__asm__ volatile ("##_ANNOTATION_INITIALREGION_-TIG-Iz-V2zB--0");
_TIG_Iz_V2zB_argc = _TIG_Iz_V2zB_formal_argc;
_TIG_Iz_V2zB_argv = _TIG_Iz_V2zB_formal_argv;
_TIG_Iz_V2zB_envp = _TIG_Iz_V2zB_formal_envp;
break;
}
_TIG_EL_V2zB_3_stringEncoder(0, _TIG_EL_V2zB_3_encodeStrings_litStr0);
#line 62
printf((char const /* __restrict */)(_TIG_EL_V2zB_3_encodeStrings_litStr0));
_TIG_EL_V2zB_3_stringEncoder(1, _TIG_EL_V2zB_3_encodeStrings_litStr1);
#line 63
scanf((char const /* __restrict */)(_TIG_EL_V2zB_3_encodeStrings_litStr1), command);
_TIG_EL_V2zB_3_stringEncoder(2, _TIG_EL_V2zB_3_encodeStrings_litStr2);
#line 64
realcommand = _TIG_EL_V2zB_3_encodeStrings_litStr2;
#line 65
tmp = strcmp((char const *) (command), (char const *) realcommand);
#line 65
if (tmp == 0L + (int) (_TIG_iE_V2zB_1_alwaysZero * (_TIG_iE_V2zB_1_entropy ^ 9) + 0)) {
_TIG_MG_V2zB_7_getUserInput_partition_printArray_quickSort_swap(0, arr, & n, n,

_TIG_Iz_V2zB_formal_argc,

0);

_TIG_EL_V2zB_3_stringEncoder(3, _TIG_EL_V2zB_3_encodeStrings_litStr3);
#line 68
printf((char const /* __restrict */)(_TIG_EL_V2zB_3_encodeStrings_litStr3));
_TIG_MG_V2zB_7_getUserInput_partition_printArray_quickSort_swap(0, 0, arr, n,

n, 2);

_TIG_MG_V2zB_7_getUserInput_partition_printArray_quickSort_swap(0, arr, 0, n - (1L +
(int) (_TIG_iE_V2zB_1_alwaysZero * (_TIG_iE_V2zB_1_entropy + 2) + 0)),

0L + (int)

(_TIG_iE_V2zB_1_alwaysZero * (_TIG_iE_V2zB_1_entropy + 2) + 0),

3);

_TIG_EL_V2zB_3_stringEncoder(4, _TIG_EL_V2zB_3_encodeStrings_litStr4);

```

```
#line 73
printf((char const /* __restrict */)(_TIG_EL_V2zB_3_encodeStrings_litStr4));
_TIG_MG_V2zB_7_getUserInput_partition_printArray_quickSort_swap(0, 0, arr, n,
                                                                    n, 2);

} else {
    _TIG_EL_V2zB_3_stringEncoder(5, _TIG_EL_V2zB_3_encodeStrings_litStr5);
#line 76
printf((char const /* __restrict */)(_TIG_EL_V2zB_3_encodeStrings_litStr5));
}
#line 79
return (0L + (int )(_TIG_iE_V2zB_1_alwaysZero * (_TIG_iE_V2zB_1_entropy - 7) + 0));
}
}
/* END FUNCTION-DEF main LOC=UNKNOWN */
```

Task 2: 反混淆技术实践 (60%)

1. 为了进入代码核心逻辑，输入需要满足什么约束 (20%)

运行与验证

```
gcc -o run_task2 Lab6_Task_2_23307130447.c -w
# 正确前缀 + 表达式示例
printf 'sozMgcLX1+2*3\n' | ./run_task2
# 错误前缀（无输出）
printf 'wrong\n' | ./run_task2
```

```
# pore @ pore in ~/pore26/pore_23307130447/Lab6 on git:master x [12:58:32]
$ gcc -o run_task2 Lab6_Task_2_23307130447.c -w

# pore @ pore in ~/pore26/pore_23307130447/Lab6 on git:master x [13:00:54]
$ printf 'sozMgcLX1+2*3\n' | ./run_task2
结果: 9.00

# pore @ pore in ~/pore26/pore_23307130447/Lab6 on git:master x [13:01:27]
$ printf 'wrong\n' | ./run_task2
```

约束说明

1. 输入方式为一行文本（`main` 中使用 `fgets` 读入缓冲区 `f[256]`）。
2. “口令”前缀：程序将常量串 `"c296TwdjTFg="` 传入函数 `g` 做 **Base64** 解码，再与输入行的前 8 个字符做 `strncmp(..., 8)`。
 - Python 验证：`base64.b64decode('c296TwdjTFg')== b'sozMgcLX'`（8 字节）。
 - 因此，**stdin 一行必须以 ASCII 字符串 `sozMgcLX` 开头**（8 个字符），否则不会进入求值与打印分支。
3. 行尾换行会被 `strcspn` 去掉，不参与后续求值。
4. 前缀之后的剩余部分由函数 `e` 解析为算术表达式（见下一节）；若仅输入 8 字符前缀而无后续合法表达式，

行为取决于 `e` 对空串/无数字的处理（得到 `0.00`）。

2. 这段代码的功能是什么，以及详细的推理过程（40%）

运行与验证（功能理解）

```
# 左结合: 与 (1+2)*3 = 9 一致
printf 'sozMgcLX1+2*3\n' | ./run_task2
# 减法
printf 'sozMgcLX10-4\n' | ./run_task2
```

```
# pore @ pore in ~/pore26/pore_23307130447/Lab6 on git:master x [13:01:46]
$ printf 'sozMgcLX\n' | ./run_task2
结果: 0.00

# pore @ pore in ~/pore26/pore_23307130447/Lab6 on git:master x [13:02:42]
$ printf 'sozMgcLX1+2*3\n' | ./run_task2
结果: 9.00

# pore @ pore in ~/pore26/pore_23307130447/Lab6 on git:master x [13:03:21]
$ printf 'sozMgcLX10-4\n' | ./run_task2
结果: 6.00
```

a) 代码功能

- `g + hh`: 见上一节引用；作用是把 Tigress **EncodeLiterals** 风格写进源码的 Base64 字面量还原成字节串，供 `strncmp` 与输入行比对。
- `main`: 逻辑见上；只有通过前缀校验后才调用 `e` 并打印结果。
- `add_obfuscated` / `mul_obfuscated` / `sub_obfuscated`: 在 `e` 的状态机里分别对应 `+`、`*`、`-` 的折叠（见 `case 15UL / 10UL / 2UL`）。实现为按位公式，属 **EncodeArithmetic** 典型形态：

```
int mul_obfuscated(int x , int y )
{
    return ((x & y) * (x | y) + (x & ~ y) * (~ x & y));
}
```

```
int add_obfuscated(int x , int y )
{
    return ((x ^ y) + ((x & y) << 1));
}
```

```
int sub_obfuscated(int x , int y )
{
    return ((x & ~ y) - (~ x & y));
}
```


4. **e**: **Flatten** 后的 `while(1){ switch(_TIG_FN_0WTT_1_e_next) }`。初始化累加器与“当前运算符”`b`（默认 `'+'`），按索引 `i` 遍历字符串：

```
case 33UL:
a = 0;
b = (char)'+';
c = 0;
has_number = 0;
expression_started = 0;
i = 0;
{
_TIG_FN_0WTT_1_e_next = 20UL;
}
```

读当前字符并判断是否为数字（`*tmp` 来自 `__ctype_b_loc()`，位 2048 对应 `isdigit`）：

```
case 20UL:
ch = (unsigned char)*(d + i);
tmp = __ctype_b_loc();
{
_TIG_FN_0WTT_1_e_next = 1UL;
}
```

```
case 1UL: ;
if (((int const) )*(tmp + (int)ch) & 2048) {
_TIG_FN_0WTT_1_e_next = 8UL;
} else {
_TIG_FN_0WTT_1_e_next = 25UL;
}
```

拼整数（十进制）：

```
case 8UL:
c = c * 10 + ((int)ch - 48);
has_number = 1;
expression_started = 1;
```

根据上一运算符 `b` 的 ASCII 值分派：`*`=42、`-`=45、`+`=43，并调用上述三个 `*_obfuscated`：

```
case 4UL: ;
switch ((int)b) {
case 42:
_TIG_FN_0WTT_1_e_next = 10UL;
break;
case 45:
_TIG_FN_0WTT_1_e_next = 2UL;
break;
case 43:
_TIG_FN_0WTT_1_e_next = 15UL;
break;
```

```
case 15UL:
a = add_obfuscated(a, c);
```

```
case 10UL:
a = mul_obfuscated(a, c);
```

```
case 2UL:
a = sub_obfuscated(a, c);
```

遇空白则 `i++` 跳过 (`*tmp__0` 位 8192 对应 `isspace`) :

```
case 35UL: ;
if ((int const )>(*tmp__0 + (int )ch) & 8192) {
    _TIG_FN_0WTT_1_e_next = 18UL;
} else {
    _TIG_FN_0WTT_1_e_next = 37UL;
}
```

读到串尾 `ch == 0` 时返回当前累加器 `a` :

```
case 3UL: ;
return ((double )a);
```

5. **综合**: 通过前缀校验后, 对**整行**做从左到右扫描的整数与 `+ - *` 运算 (无优先级), 结果以 `double` 打印。

b) 推理过程

1. **定位入口**: 7042-7044 行显示口令为 编码后的字面量 经 `g` 解码再 `strcmp`; 结合 7143-7191 的移位与 `hh` 字母表, 可判定为 Base64; 解码得到 8 字节前缀 `sozMgcLX`。

```
7042 | tmp__1 = g((char const *) "c296TWdjTFg=");
7043 | #line 126
7044 | tmp__2 = strcmp((char const *) (f), (char const *) tmp__1, (size_t) 8);
```

```

7143     c = (b / 4) * 3;
7144 #line 24
7145     if ((int const  )(a + (b - 1)) == 61) {
7146 #line 24
7147         c --;
7148     }
7149 #line 25
7150     if ((int const  )(a + (b - 2)) == 61) {
7151 #line 25
7152         c --;
7153     }
7154 #line 27
7155     tmp__0 = malloc((size_t )(c + 1));
7156 #line 27
7157     d = (char *)tmp__0;
7158 #line 28
7159     i = 0;
7160 #line 28
7161     j = 0;
7162 #line 29
7163     e__0 = 0;
7164 #line 29
7165     f = 0;
7166 #line 30
7167     while (i < b) {
7168 #line 31
7169         tmp__1 = i;
7170 #line 31
7171         i ++;
7172 #line 31
7173         c__0 = (char )(a + tmp__1);
7174 #line 32
7175         if ((int )c__0 == 61) {
7176 #line 32
7177             break;
7178         }
7179 #line 34
7180         tmp__2 = hh(c__0);
7181 #line 34
7182         o = tmp__2;
7183 #line 35
7184         if (o == -1) {
7185 #line 35
7186             continue;
7187         }
7188 #line 37
7189         e__0 = (e__0 << 6) | o;
7190 #line 38
7191         f ++;

```

2. 识别 **EncodeArithmetic**: 7064 - 6937 三段返回式为按位恒等变形, 且在 **e** 中仅通过 **add_obfuscated** / **mul_obfuscated** / **sub_obfuscated** 更新 **a**, 可对应到普通 **+**、*****、**-**。

```

6937 | return ((x & ~ y) - (~ x & y));
6938 | }
6939 | }
6940 | /* END FUNCTION-DEF sub_obfuscated LOC=UNKNOWN */
6941 |
6942 | /* BEGIN FUNCTION-DEF __uint64_identity LOC=UNKNOWN */
6943 | __inline static __uint64_t __uint64_identity(__uint64_t __x )
6944 | {
6945 |
6946 |
6947 | {
6948 | #line 47 "/usr/include/x86_64-linux-gnu/bits/uintn-identity.h"
6949 | | return (__x);
6950 | }
6951 | }
6952 | /* END FUNCTION-DEF __uint64_identity LOC=UNKNOWN */
6953 |
6954 | /* BEGIN FUNCTION-DEF __bswap_32 LOC=UNKNOWN */
6955 | __inline static __uint32_t __bswap_32(__uint32_t __bsx )
6956 | {
6957 | | int tmp ;
6958 |
6959 | {
6960 | #line 52 "/usr/include/x86_64-linux-gnu/bits/byteswap.h"
6961 | | tmp = __builtin_bswap32((int )__bsx);
6962 | #line 52
6963 | | return ((__uint32_t )tmp);
6964 | }
6965 | }
6966 | /* END FUNCTION-DEF __bswap_32 LOC=UNKNOWN */
6967 |
6968 | /* BEGIN FUNCTION-DEF __uint32_identity LOC=UNKNOWN */
6969 | __inline static __uint32_t __uint32_identity(__uint32_t __x )
6970 | {
6971 |
6972 |
6973 | {
6974 | #line 41 "/usr/include/x86_64-linux-gnu/bits/uintn-identity.h"
6975 | | return (__x);
6976 | }
6977 | }
6978 | /* END FUNCTION-DEF __uint32_identity LOC=UNKNOWN */
6979 |
6980 | /* BEGIN FUNCTION-DEF __bswap_64 LOC=UNKNOWN */
6981 | __inline static __uint64_t __bswap_64(__uint64_t __bsx )

```

```

6981 __inline static __uint64_t __bswap_64(__uint64_t __bsx )
6982 {
6983     long tmp ;
6984
6985     {
6986 #line 73 "/usr/include/x86_64-linux-gnu/bits/byteswap.h"
6987     tmp = __builtin_bswap64((__long )__bsx);
6988 #line 73
6989     return ((__uint64_t )tmp);
6990 }
6991 }
6992 /* END FUNCTION-DEF __bswap_64 LOC=UNKNOWN */
6993
6994 /* BEGIN FUNCTION-DEF main LOC=UNKNOWN */
6995 int main(int _TIG_IZ_0WTT_formal_argc , char **_TIG_IZ_0WTT_formal_argv , char **_TIG_IZ_0WTT
6996 {
6997     char f[256] ;
6998     size_t tmp ;
6999     double a ;
7000     double tmp__0 ;
7001     char *tmp__1 ;
7002     int tmp__2 ;
7003
7004     {
7005     {
7006     {
7007     {
7008     {
7009         _TIG_IZ_0WTT_envp = (char **)0;
7010         goto _TIG_IZ_0WTT_envp__INITINLINE__TIG_IZ_0WTT_envp__INIT;
7011     }
7012     _TIG_IZ_0WTT_envp__INITINLINE__TIG_IZ_0WTT_envp__INIT: /* CIL Label */ ;
7013     }
7014     {
7015     {
7016         _TIG_IZ_0WTT_argv = (char **)0;
7017         goto _TIG_IZ_0WTT_argv__INITINLINE__TIG_IZ_0WTT_argv__INIT;
7018     }
7019     _TIG_IZ_0WTT_argv__INITINLINE__TIG_IZ_0WTT_argv__INIT: /* CIL Label */ ;
7020     }
7021     {
7022     {
7023         _TIG_IZ_0WTT_argc = 0;
7024         goto _TIG_IZ_0WTT_argc__INITINLINE__TIG_IZ_0WTT_argc__INIT;
7025     }
7026     _TIG_IZ_0WTT_argc__INITINLINE__TIG_IZ_0WTT_argc__INIT: /* CIL Label */ ;
7027     }

```

```

7027     }
7028     goto megaInitINLINE_megaInit;
7029 }
7030 megaInitINLINE_megaInit: /* CIL Label */ ;
7031 }
7032 while (1) {
7033     __asm__ volatile ("##_ANNOTATION_INITIALREGION_-TIG-IZ-0WTT--0");
7034     _TIG_IZ_0WTT_argc = _TIG_IZ_0WTT_formal_argc;
7035     _TIG_IZ_0WTT_argv = _TIG_IZ_0WTT_formal_argv;
7036     _TIG_IZ_0WTT_envp = _TIG_IZ_0WTT_formal_envp;
7037     break;
7038 }
7039 #line 125 "/home/pore/Lab6/students/23307130447/Lab6_Task_2_origin_23307130447.c"
7040 fgets((char /* __restrict */)(f), (int )sizeof(f), (FILE /* __restrict */)(stdin);
7041 #line 126
7042 tmp__1 = g((char const *) "c296TWdjTFg=");
7043 #line 126
7044 tmp__2 = strncmp((char const *)(f), (char const *)tmp__1, (size_t )8);
7045 #line 126
7046 if (tmp__2 == 0) {
7047 #line 128
7048     tmp = strcspn((char const *)(f), (char const *) "\n");
7049 #line 128
7050     f[tmp] = (char)0;
7051 #line 130
7052     tmp__0 = e((char const *)(f));
7053 #line 130
7054     a = tmp__0;
7055 #line 131
7056     printf((char const /* __restrict */)"\347\273\223\346\236\234: %.2f\n", a);
7057 }
7058 #line 133
7059 return (0);
7060 }
7061 }
7062 /* END FUNCTION-DEF main LOC=UNKNOWN */
7063
7064 /* BEGIN FUNCTION-DEF mul_obfuscated LOC=UNKNOWN */

```

3. 还原 e 的算法:

- 6677-6678 行 switch (_TIG_FN_0WTT_1_e_next) 为 Flatten 控制流;

```

6677     while (1) {
6678         switch (_TIG_FN_0WTT_1_e_next) {

```

- 6851-6861 行初始化 a、b='+'、c、i;

```

6850     #line 73
6851     a = 0;
6852     #line 74
6853     b = (char) '+';
6854     #line 75
6855     c = 0;
6856     #line 76
6857     has_number = 0;
6858     #line 77
6859     expression_started = 0;
6860     #line 79
6861     i = 0;

```

- 6915 行按 d[i] 逐字符处理;

```
6914    #line 80
6915    |      | ch = (unsigned char)*(d + i);
```

- 6699-6714 行将 `b` 与 `*`、`+`、`-` 绑定到 10/15/2 分支，即先按上一运算符合并 `a` 与当前数 `c`，再读入新运算符（6763-6771 等），故为从左到右、无先乘除后加减。

```
6699    switch ((int)b) {
6700    case 42:
6701    {
6702        _TIG_FN_0WTT_1_e_next = 10UL;
6703    }
6704    break;
6705    case 45:
6706    {
6707        _TIG_FN_0WTT_1_e_next = 2UL;
6708    }
6709    break;
6710    case 43:
6711    {
6712        _TIG_FN_0WTT_1_e_next = 15UL;
6713    }
6714    break;
```

4. 验证求值顺序：

- `1+2*3`：先算 `0+1`，再 `(1)+2` 得 3，再 `3*3` 得 9（对应 `mul` 在第三次折叠），与运行 9.00 一致；若存在优先级应先算 `2*3` 得 7。
- `10-4`：`sub_obfuscated(10,4)` → 6.00。

故结论：严格从左到右、无运算符优先级。

实验分析与总结

- **Merge 与 main**：Tigress 将 `main` 列入 Merge 的 `--Functions` 时在本版本失败，采用手册给出的五个函数合并，逻辑上仍覆盖快排核心路径。
- **任务 2 的“双重混淆”**：字符串层（Base64/EncodeLiterals 思路）与控制流/算术层（Flatten + EncodeArithmetic）叠加，静态阅读需分层剥离：先常量再算法。
- **自动化验证**：对所有任务 1 生成文件执行统一输入 `quicksort + 3 + 3 1 2`，均得到正确排序，说明混淆保持语义。